Multi-Threaded Hospital Emergency Management System

Problem Statement :-

Hospital emergency departments often receive many patients at the same time, each with different medical conditions. These patients need **immediate and proper medical care**. If patient handling is not organized, it may lead to treatment delays, which can be dangerous in critical cases.

The aim of this project is to develop a **Hospital Emergency Management System using Java and Object-Oriented Programming (OOP)** concepts. The system will simulate how an emergency department manages incoming patients and provides medical treatment.

When a patient arrives, their information will be recorded and they will be placed in a **queue for treatment**. The doctor will examine the patient and provide the appropriate medical care based on the patient's condition.

To represent real-life scenarios where many patients arrive at once, **multi-threading** will be used. The **Java Queue collection** will help manage the order in which patients are treated.

Important OOP principles such as **encapsulation** will protect patient data, and **polymorphism** will allow the system to provide different treatments depending on the patient's condition.

The main objective of this project is to demonstrate how **Java OOP concepts can simulate a real hospital emergency management system.**

Target Users / Audience :-

The system can be useful for:

- Hospital emergency department staff
- Doctors and nurses
- Hospital management
- Medical trainees
- Students learning **Java and Object-Oriented Programming**

Core Features :-

The system will provide the following functionalities:

- Register patients with **name, age, and medical condition**.
- Simulate **multiple patient arrivals using threads**.
- Maintain an **emergency patient queue using Java Collections**.

- Allow doctors to check the patient's medical condition.
- Select and provide appropriate treatments such as:
    - Medicine
    - First Aid
    - Surgery
- Display patient information and treatment details.
- Handle incorrect or invalid inputs using **exception handling**.

Unique Medical Features (Important in Real Hospitals) :-

To make the system more practical for healthcare environments, the following unique features are included:

1. Emergency Priority System

- Patients will be assigned **priority levels** such as:
    - Critical
    - Serious
    - Normal
- Critical patients will be treated **before others**, even if they arrived later.

2. Patient Vital Monitoring (NOW ,I PUT IN MANUALLY)

- The system will store basic **vital signs** such as:
    - Heart rate
    - Blood pressure
    - Temperature
- This helps doctors quickly assess patient conditions.

3. Patient Medical Record

- The system will keep a **basic treatment history** of patients for reference.

OOP Concepts Used :-

The project uses several Object-Oriented Programming concepts:

- **Abstraction** – hides system complexity
- **Inheritance** – treatment classes inherit from a base class
- **Polymorphism** – different treatments depending on condition
- **Encapsulation** – protects patient data

- **Exception Handling** – manages errors and invalid inputs
- **Collections Framework** – manages the emergency queue
- **Multi-Threading** – simulates simultaneous patient arrivals

System Architecture Overview :-

The system will consist of multiple classes working together.

Patient Class

- Stores patient information such as **name, age, condition, and vital signs**.
- Uses encapsulation to protect patient data.

Doctor Class

- Retrieves patients from the queue.
- Checks the patient condition and decides the treatment.

Treatment Classes

- Includes classes such as:
    - **Medicine**
    - **FirstAid**
    - **Surgery**
- Demonstrates **inheritance and polymorphism**.

EmergencyPriority Class

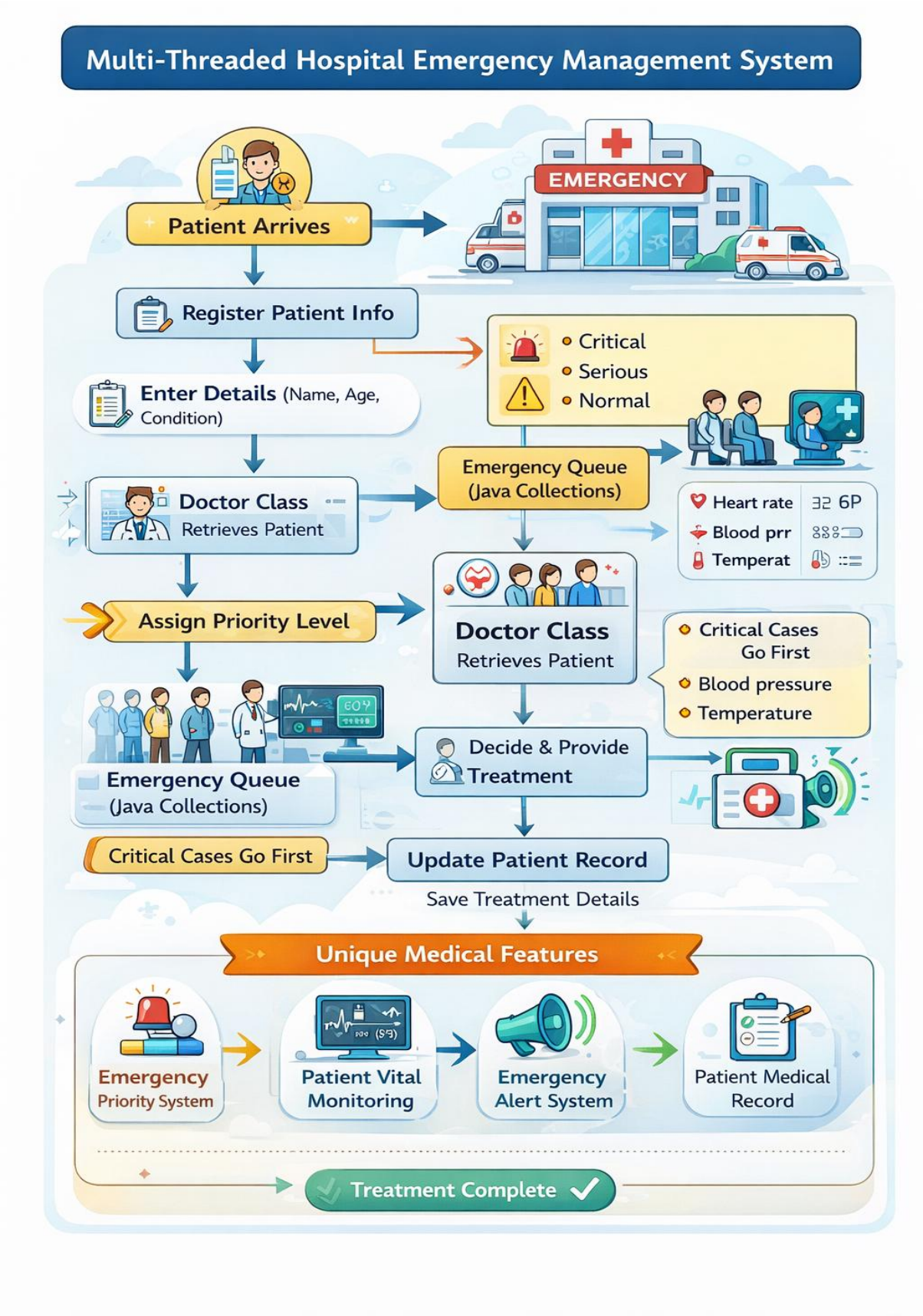
- Determines the **priority level of patients** based on condition severity.

HospitalSystem Class

- Controls the main program execution.
- Manages communication between **patients, doctors, treatments, and queues**.

Patients will be generated as **threads**, and each patient will be added to the **emergency queue using Java Collections**, allowing the doctor to treat them efficiently.

FLOW CHART


Multi-Threaded Hospital Emergency Management System
Patient Arrives
EMERGENCY
Register Patient Info
Enter Details (Name, Age, Condition)
Critical
Serious
Normal
Emergency Queue (Java Collections)
Doctor Class Retrieves Patient
Heart rate
Blood prr
Temperat
Assign Priority Level
Doctor Class Retrieves Patient
Critical Cases Go First
Blood pressure
Temperature
Emergency Queue (Java Collections)
Decide & Provide Treatment
Critical Cases Go First
Update Patient Record
Save Treatment Details
Unique Medical Features
Emergency Priority System
Patient Vital Monitoring
Emergency Alert System
Patient Medical Record
Treatment Complete

Conclusion :-

The **Multi-Threaded Hospital Emergency Management System** demonstrates how Java and Object-Oriented Programming concepts can be applied to simulate a real-world hospital emergency environment. The system efficiently manages patient registration, prioritization, and treatment using features such as multi-threading, collections, and various OOP principles.

By using a queue-based system and priority levels, the application ensures that patients with critical conditions receive treatment first. The system also records important patient information, vital signs, and treatment history, which helps doctors make quick and informed decisions.

This project not only improves understanding of **Java programming concepts such as encapsulation, inheritance, polymorphism, abstraction, exception handling, and multi-threading**, but also shows how these concepts can be used to solve real-life problems in healthcare management.

Overall, the system provides a basic yet effective simulation of how hospital emergency departments can manage multiple patients efficiently and deliver timely medical care.

Thank You :-

Finally, I would like to sincerely thank all the readers for taking the time to go through this project report. Your interest, patience, and willingness to understand this work are truly appreciated.

This project was created not only to demonstrate programming concepts but also to explore how technology can be used to solve real-world problems in the healthcare sector. I hope this report has provided useful insights into how Java and Object-Oriented Programming can be applied to build practical systems such as a Hospital Emergency Management System.

Your time and attention mean a lot, and any feedback or suggestions for improvement would be greatly valued.

Thank you very much for reading and supporting this work.